Technical Documentation

NATS Messaging System

In-Depth JWT Guide for NATS

A Comprehensive Technical Deep Dive into Decentralized Authentication & Authorization

CHAIN OF TRUST

DECENTRALIZED SECURITY

Version 2.0 | 2026

PRESENTATION OVERVIEW

Contents

01 Core Concepts

Understanding Accounts, NKEYs, and JWT Fundamentals in NATS

02 Decentralized Architecture

Chain of Trust, Verification & Deployment Models

03 Tooling & Key Management

NSC CLI, Environment Setup & Security Best Practices

04 Advanced Topics

System Account, Connectivity & Key Lifecycle Management

Security Focus

Decentralized authentication with hierarchical chain of trust

Practical Examples

Code snippets, configuration examples & deployment scenarios

01

Core Concepts

Understanding Accounts, NKEYs, and JWT Fundamentals

Accounts | NKEYs | JWT

CORE CONCEPTS

What Are Accounts?

Accounts are the NATS isolation context. Messages published in one account won't be received in another unless explicitly defined through imports and exports.

Configuration Example

```
accounts: {
    A: {
        users: [{user: a, password: a}]
    },
    B: {
        users: [{user: b, password: b}]
    },
}
```

Messages in account A are isolated from account B

Cross-Account Communication

```
accounts: {
    A: {
        users: [{user: a, password: a}]
        imports: [{stream: {account: B, subject: "foo"}}]
    },
    B: {
        users: [{user: b, password: b}]
        exports: [{stream: "foo"}]
    },
```

🔑 Key Takeaways

- **Accounts are isolated** from each other by default
- **Selective combination** possible via imports/exports
- **Config file changes** required for modifications
- Apply changes via `nats-server --signal reload`

Account Scope

Per Team
One account per organizational team

Per Application
Fine-grained, service-level isolation

Recommendation
Scope by application/service offered

CORE CONCEPTS

NKEYs: Secure Authentication Foundation

NKEYs are decorated, Base32 encoded, CRC16 check-summed Ed25519 keys.
Ed25519 is a public key signature system resistant to side channel attacks.

Key Types & Prefixes

- **O** — **Operator**
  Public: O... I Seed: SO...
- **A** — **Account**
  Public: A... I Seed: SA...
- **U** — **User**
  Public: U... I Seed: SU...

NKEY Authentication Flow

1. **Server sends challenge** (nonce) in INFO message
2. **Client signs challenge** with private NKEY
3. **Server verifies signature** with public NKEY

Private key never leaves the client

Key Takeaways

- **NKEYs are secure** way to authenticate clients
- **Private keys never accessed** or stored by NATS server
- **Public key configured** in NATS server

Key Generation

```
nk -gen user -pubout > a.nk
cat a.nk
SUAAEZYNLTEA2MDTG7L5X7QODZXYHPOI2LT2KH5I4GD6YVP24SE766EGPA
UC435ZYS52HF72E2VMQF4GO6CUJOCHDUUPEBU7XDXW5AQLIC6JZ46PO5
```

Usage in NATS Config

```
users: [{
  nkey: UC435ZYS52HF72E2VMQF4GO6CUJOCH...
}]
```

Connect: nats sub --nkey=a.nk ">"

CORE CONCEPTS

JSON Web Tokens (JWT) in NATS

Motivation for JWT

In large organizations, centralized configuration can lead to less flexibility and resistance to change. JWT splits nats-server configuration into separate artifacts manageable by different entities.

Separation of Concerns

Infrastructure
NATS servers, clusters, topology

Accounts
Teams, applications, services

Users
Individual clients, applications

🔑 Key Takeaways

- **JWT splits configuration** into separate artifacts
- **Management separated:** Accounts, Config, Users
- **Accounts correspond to teams/applications**, not infrastructure
- **Connect to any cluster** in same infrastructure
- **Infrastructure topology** independent of accounts

JWT Benefits

Decentralized
Independent management

Scalable
Handle huge account volumes

Secure
Hierarchical trust chain

Flexible
Multiple deployment models

02

Decentralized Architecture

Chain of Trust, Verification & Deployment Models

HIERARCHY | VERIFICATION | DEPLOYMENT

DECENTRALIZED ARCHITECTURE

JWT Hierarchy & Chain of Trust

Account and User creation managed as separate artifacts using NKEYs, relying on a hierarchical chain of trust between three distinct NKEYs.

Three-Level Hierarchy

O **Operator**
Controls NATS servers in authentication domain

A **Account**
Single account's configuration & limits

U **User**
Individual user's permissions & limits

JWT Structure

```
{
  "sub": "public-nkey-identity",
  "iss": "issuer-public-key",
  "iat": 1603473819,
  "type": "operator|account|user",
  "nats": { /* role-specific config */ }
}
```

🔑 Key Takeaways

- **JWTs are hierarchically organized** in operator, account, user
- **Each carries role-specific configuration**
- **Issuer field establishes trust** relationships
- **Decentralized management** at each level

JWT Content by Role

Operator JWT

- Server configuration
- System account reference
- Account server URL

Account JWT

- Exports & imports
- Limits & constraints
- Default permissions

User JWT

- User permissions
- Connection limits
- Time restrictions

DECENTRALIZED ARCHITECTURE

Chain of Trust Verification Process

Complete Verification Flow

1. **Server Configuration**
   NATS server trusts operator JWT; configured with resolver
2. **Client Connection**
   Client provides User JWT + signed nonce proving NKEY possession
3. **Three-Level Verification**
   User JWT → Account JWT → Operator JWT (cascading validation)

Verification Steps

- ✓ Validate User JWT signature
- ✓ Verify nonce matches JWT.subject
- ✓ Obtain & validate Account JWT
- ✓ Verify Account issuer is trusted Operator
- ✓ Check User not in revocation list
- ✓ Verify User issuer matches Account

Trust Rules

Type	Trust Rule
Operator	jwt.issuer == jwt.subject (self-signed)
Account	jwt.issuer == trusted operator key
User	jwt.issuer == trusted account key && not revoked

Decoded JWT Example

```
{
  "iat": 1603473819,
  "iss": "OBU5O5FJ324...",
  "sub": "OBU5O5FJ324...",
  "type": "operator",
  "name": "DEMO"
}
```

Fields: iat (issued at), iss (issuer), sub (subject)

DECENTRALIZED ARCHITECTURE

Account Resolver Types

The nats-server is configured with one of three resolver types to obtain Account JWTs. The choice depends on your operational needs and scale.

Resolver Comparison

mem-resolver

- Very few or static accounts
- Requires server reload on changes
- JWTs embedded in config file

url-resolver

- Very large volume of accounts
- Downloads from web server
- nats-account-server compatible

⭐ nats-resolver ⭐ RECOMMENDED

- **Same as url-resolver**, uses NATS protocol
- **No separate binary** to run/config/monitor
- **Easier clustering** - eventually consistent
- **Supports account removal** (with allow_delete)
- **Production recommended** for large deployments

Configuration Example

```
operator: ./trustedOperator.jwt
resolver: URL(http://localhost:9090/jwt/v1/accounts/)
# OR for nats-resolver:
resolver: nats://localhost:4222
```

Recommendation: Start with mem-resolver for learning, then migrate to nats-resolver for production

DECENTRALIZED ARCHITECTURE

Deployment Models Enabled by Chain of Trust

Different deployment models are enabled based on which entity has access to private Operator/Account identity or signing NKEYs. **Pick the simplest model that fits your needs.**

Model 1: Centralized Config

Single Entity Control
One user/set has access to ALL private operator and account NKEYs

Best for: Small teams, single organization, simple governance

Model 2: Decentralized Config

Split Responsibilities
Administrators control operator; teams control their accounts

Best for: Multiple teams, shared infrastructure, controlled autonomy

Model 3: Self-Service

Team Autonomy
Teams have operator signing key; can create/modify accounts

⚠ **Requires:** Organizational mechanism to prevent unchecked usage

Best for: Dev clusters, rapid prototyping, mature organizations

Key Principles

- **Pick the simplest model** that enables what you need
- **Signing keys enable flexibility** without compromising identity keys
- **Sign-up services** can be implemented with signing keys

03

Tooling & Key Management

NSC CLI, Environment Setup & Security Best Practices

NSC CLI | ENVIRONMENT | SECURITY

NSC Environment & Directory Structure

nsc is a tool that uses JWT and NKEY libraries to create NKEYs and all types of JWT. It stores these artifacts in separate directories for security and organization.

Directory Structure

NKEYS Store Directory

Contains private keys (seeds) - **HIGHLY SENSITIVE**

```
~/.nkeys/keys/O/DM/...
~/.nkeys/keys/A/DR/...
~/.nkeys/keys/U/C4/...
```

JWT Store Directory

Contains JWTs - **NO PRIVATE KEYS**

```
~/.nsc/nats/DEMO/DEMO.jwt
~/.nsc/nats/DEMO/accounts/...
```

Environment Commands

View Environment

```
nsc env
```

Switch JWT Store

```
nsc env --store
```

Switch NKEY Store

```
export NKEYS_PATH=
```

Backup Strategy

- **NKEYs:** Offline backup, severely restricted access
- **Identity keys:** Archive offline, remove from store
- **JWTs:** Can be in VCS (no private keys)
- **Restore:** Check expiration, revocations, key presence

Setting Up Operator & Account

Operator Setup

Create Operator with System Account

```
nsc add operator -n DEMO --sys
```

Edit Operator

```
nsc edit operator --account-jwt-server-url "nats://localhost:4222"
```

Generate Signing Key

```
nsc generate nkey -o --store
nsc edit operator --sk O...
```

Account Setup

Create Account

```
nsc add account -n  -i
```

Edit Account

```
nsc edit account [flags]
```

Key Listing Example

```
+-----------------------------------------------------+
|                    Keys                             |
+--------+-----------------------------------+----------+
| Entity | Key                               | Signing  |
+--------+-----------------------------------+----------+
| DEMO   | OD5FHU4LXGDSGDHO7UNRMLW6I36...    |          |
| DEMO   | OBYAIG4T4PVR6GVYDERN74RRW7V...    | *        |
|  ACC   | ADRB4JJYFDLWKIMX4DH6MX2DMKA...    |          |
|  SYS   | AAYVLZJC2ULKSH5HNSKMIKFMCEH...    |          |
|   sys  | UBVZYLLCAFMHBXBUDKKKFKH62T4...    |          |
+--------+-----------------------------------+----------+
```

* = signing key | Stored column indicates key presence

Import/Export

- **Export:** `nsc describe operator --raw`
- **Import:** `nsc add operator -u operator.jwt`
- **Update:** Use `--force` to overwrite

User Creation & Automated Sign-Up Services

Manual User Creation

Create User

```
nsc add user --account  --name  -i
```

Edit User

```
nsc edit user [flags]
```

Distributed User Creation

1. Requester Generates NKEY

```
nsc generate nkey -u
```

(omit --store for security)

2. Exchange Public Key

Securely exchange public key (not private!)

3. Create User with Public Key

```
nsc add user -a  -n  -k
```

Programmatic User Generation (Go)

```go
func generateUserJWT(userPublicKey,
    accountPublicKey string,
    accountSigningKey nkeys.KeyPair) string {

    uc := jwt.NewUserClaims(userPublicKey)
    uc.Pub.Allow.Add("subject.foo")
    uc.Expires = time.Now().Add(time.Hour).Unix()
    uc.IssuerAccount = accountPublicKey

    userJWT, _ := uc.Encode(accountSigningKey)
    return userJWT
}
```

⚠ Security Best Practices

- **Set SHORTEST possible expiration** for sign-up JWTs
- **Properly authenticate** incoming requests
- **Use signing keys** for dynamic generation

04

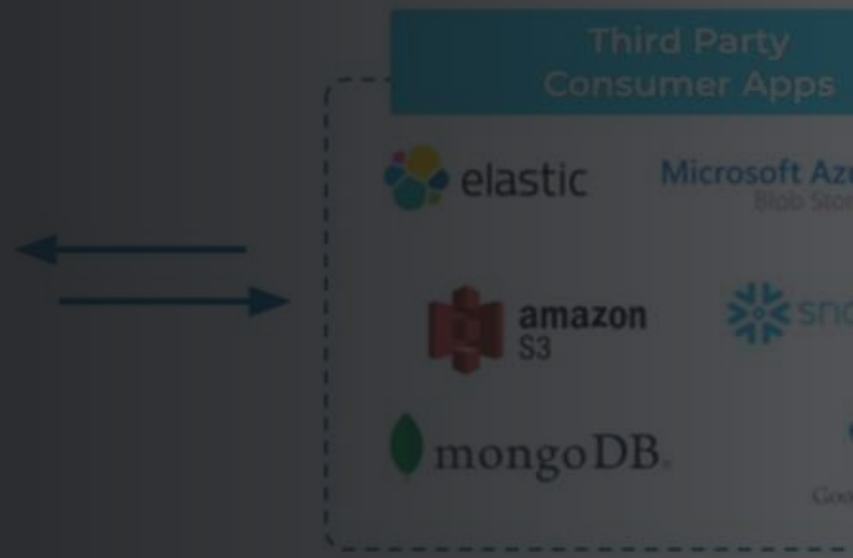
Advanced Topics

System Account, Connectivity & Key Lifecycle Management

SYSTEM ACCOUNT

EXPORTS/IMPORTS

REVOCATION



ADVANCED TOPICS

System Account & Monitoring Services

The system account is the account under which nats-server offers administrative services and monitoring events. NOT for application communication.

Event Subjects

Subject	Description
$SYS.SERVER.*.SHUTDOWN	Server shutdown
$SYS.SERVER.*.CLIENT.AUTH.ERR	Auth failure
$SYS.SERVER.*.STATSZ	Stats (periodic)
$SYS.ACCOUNT.*.CONNECT	Client connected
$SYS.ACCOUNT.*.DISCONNECT	Client disconnected
$SYS.ACCOUNT.*.SERVER.CONNS	Conns changed

Service Subjects

Subject	Description
$SYS.REQ.SERVER.PING.STATZ	Server stats
$SYS.REQ.SERVER.PING.CONNZ	Connections
$SYS.REQ.SERVER.PING.VARZ	Variables
$SYS.REQ.SERVER.PING.SUBZ	Subscriptions
$SYS.REQ.ACCOUNT.*.INFO	Account info
$SYS.REQ.ACCOUNT.*.CONNS	Account conns

NATS Resolver Subjects

$SYS.REQ.CLAIMS.UPDATE
$SYS.REQ.ACCOUNT.*.CLAIMS.UPDATE
$SYS.REQ.ACCOUNT.*.CLAIMS.LOOKUP
$SYS.REQ.CLAIMS.PACK

ADVANCED TOPICS

Connecting Accounts: Exports & Imports

Accounts connect through exports and imports. Exports define what an account exposes; imports define what an account consumes. Activation tokens control access to private exports.

Export Types

Public Export

```
nsc add export --name  --subject
```

Any account can import

Private Export

```
nsc add export --name  --subject  --private
```

Requires activation token

Import Types

Public Import

```
nsc add import --account  --src-account  --remote-subject
```

Private Import (with token)

```
nsc add import --account  --token
```

Generate Activation Token

```
nsc generate activation --account  --subject  --target-account
```

Visualize Relationships

```
nsc generate diagram component --output-file test.uml
plantuml -tpng test.uml
```

ADVANCED TOPICS

Leaf Node Connections

⚠ **Important:** Leaf nodes do NOT multiplex between accounts. Every account must be explicitly listed, including the system account.

Non-Operator Mode

System account may differ from user account

```
leafnodes {
    remotes = [
        {
          url: "nats://localhost:4222"
          credentials: "./your-account.creds"
        },
        {
          url: "nats://localhost:4222"
          account: "$SYS"
          credentials: "./system-account.creds"
        },
    ]
}
```

Operator Mode

Same operator and system account used

```
operator: ./trustedOperator.jwt
system_account: AAAXAUVSGK7TCRHFIRAS4SYXVJ76...
leafnodes {
    remotes = [
        {
          url: "nats://localhost:4222"
          account: "ADKGAJU55CHYOIF5H432K2Z2ME3..."
          credentials: "./your-account.creds"
        },
        {
          url: "nats://localhost:4222"
          account: "AAAXAUVSGK7TCRHFIRAS4SYXVJ..."
          credentials: "./system-account.creds"
        },
    ]
}
```

Key Points

- Each account needs explicit remote entry
- System account user can be heavily restricted
- Required for nats-resolver and monitoring

ADVANCED TOPICS

Key Management & Revocation Strategies

Protect Identity NKEYs

Strategy: Take Offline

- Generate signing keys first
- Backup identity keys offline
- Delete from nsc NKEY directory

Key Location

```
~/.nkeys/keys/O/DM/ODMFND7E... .nk
~/.nkeys/keys/A/DR/ADRB4JJY... .nk
```

Reissue Identity NKEYs

Operator

```
nsc reissue operator
# Or convert to signing key:
nsc reissue operator --convert-to-signing-key
```

Account

- Generate new account with similar settings
- Re-sign all users
- Push account, deploy new user JWTs

Revocation Mechanisms

Revoke User

```
nsc revocations add-user \
  --account  --name
nsc revocations list-users
nsc revocations delete-user \
  --account  --name
```

Revoke Activation

```
nsc revocations add-activation \
  --account  --subject  \
  --target-account
```

Revoke Account

```
# Lock out all users
nsc edit account --name  --conns 0
nsc push --all
# Or delete account
nsc delete account --name
nsc push --all --prune
```

Remove Signing Keys (Bulk Revocation)

```
nsc edit operator --rm-sk
nsc edit account --name  --rm-sk
```

KEY TAKEAWAYS

Mastering NATS JWT

Decentralized Authentication

JWTs enable flexible, secure authentication across operators, accounts, and users with hierarchical chain of trust

Chain of Trust

Three-level verification ensures security: User → Account → Operator with cascading validation at each step

Deployment Models

Choose from centralized, decentralized, or self-service models based on organizational needs and complexity

Key Security

Protect identity keys offline, use signing keys for operational flexibility, implement proper revocation strategies

nats-resolver recommended for production | nsc for streamlined management